Voor jouw apotheek

SvelteKit Fundamentals

Form Actions

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

What are Form actions?

"Form actions are SvelteKit's built-in way to handle POST requests without writing manual fetch calls."

Instead of

```
fetch('/api/login', { method: 'POST', body: ... })
```

We write

```
<form method="POST">
```

And handle submissions on the (Svelte) server.

Might not be suitable in all applications and/or situations, but is a built-in SvelteKit way to handle form submissions easily

Form actions - Features

- Close to how the web works – uses plain HTML forms
- Works without JavaScript/TypeScript
- No boilerplate `fetch()` logic or `JSON` parsing
- Server side by default
  - E.g. Safe, easy validation, direct DB/API access from server code

Mental shift:

- *“Stop thinking in API calls, start thinking in form submissions.”*

How does it work?

- Form actions live in `+page.server.ts`
- The form itself lives in `+page.svelte`
- If the form is POSTed, by default the browser posts the form back to itself
- Then, the function in `+page.server.ts` kicks in
- It reloads the page, now with a `form` object in `$props()`
- We can now use `form.XYZ` in the page

Example +page.server.ts

```
// +page.server.ts - handling a very simple form
export const actions = {

  default: async ({ request }) => {
   const data = await request.formData();

   const name = data.get('name');

   console.log('Received:', name);

   return {
    success: true
   };
  }
};
```

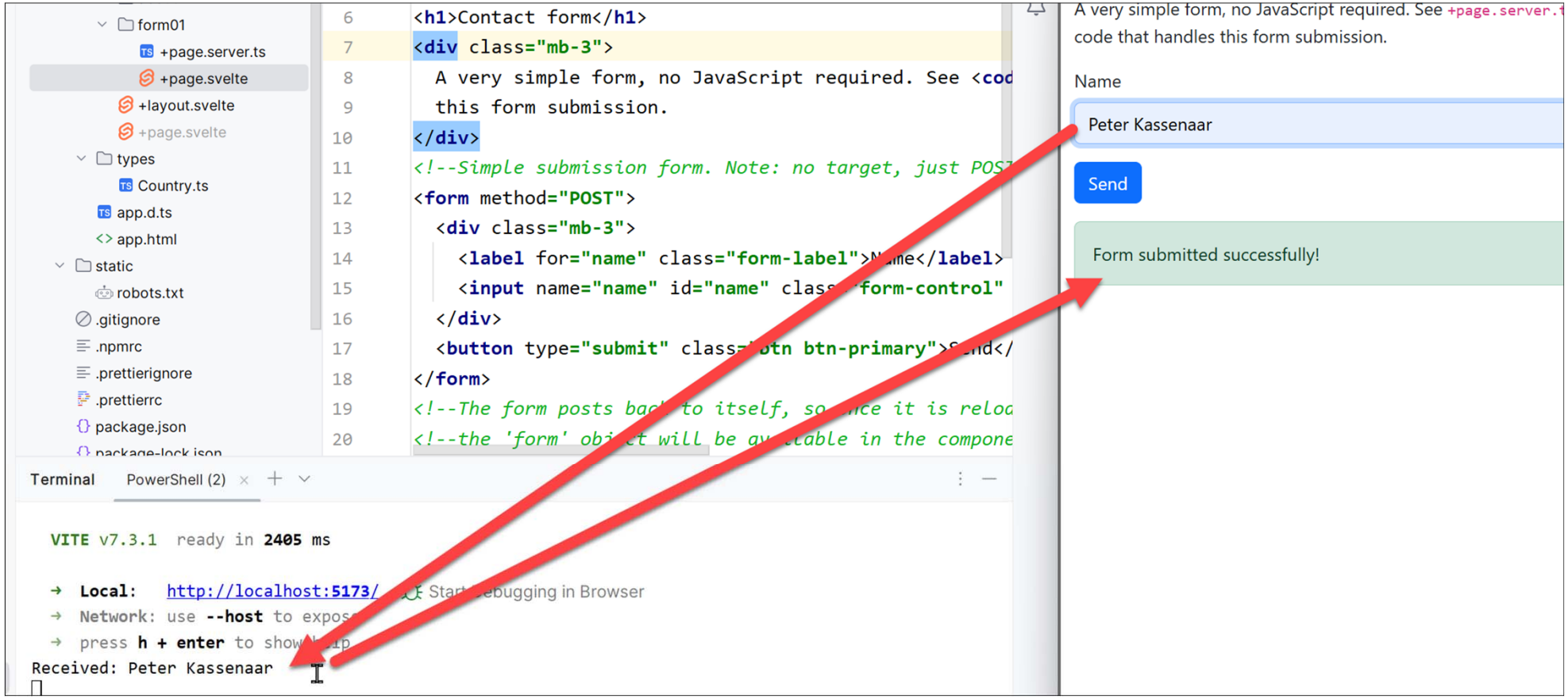
Example +page.svelte (no JavaScript)

```
<script lang="ts">
  const { form } = $props();
</script>

<h1>Contact form</h1>
…
<!--Simple submission form. Note: no target, just POST-->
<form method="POST">
   <label for="name">Name</label>
   <input name="name" id="name" placeholder="Your name" />
   <button type="submit">Send</button>
</form>

{#if form?.success}
  <div>Form submitted successfully!</div>
{/if}
```

Result


form01
+page.server.ts
+page.svelte
+layout.svelte
+page.svelte
types
Country.ts
app.d.ts
app.html
static
robots.txt
.gitignore
.npmrc
.prettierignore
.prettierrc
package.json
6 <h1>Contact form</h1>
7
8 A very simple form, no JavaScript required. See <cod
9 this form submission.
10
11 <!--Simple submission form. Note: no target, just POS
12 <form method="POST">
13
14 <label for="name" class="form-label">Name</label>
15 <input name="name" id="name" class="form-control"
16
17 <button type="submit" class="btn btn-primary">Send</
18 </form>
19 <!--The form posts back to itself, so once it is reloa
20 <!--the 'form' object will be available in the compone
Terminal
PowerShell (2)
VITE v7.3.1 ready in 2405 ms
Local: http://localhost:5173/
Start debugging in Browser
Network: use --host to expose
press h + enter to show help
Received: Peter Kassenaar
A very simple form, no JavaScript required. See +page.server.t
code that handles this form submission.
Name
Peter Kassenaar
Send
Form submitted successfully!

Learning points

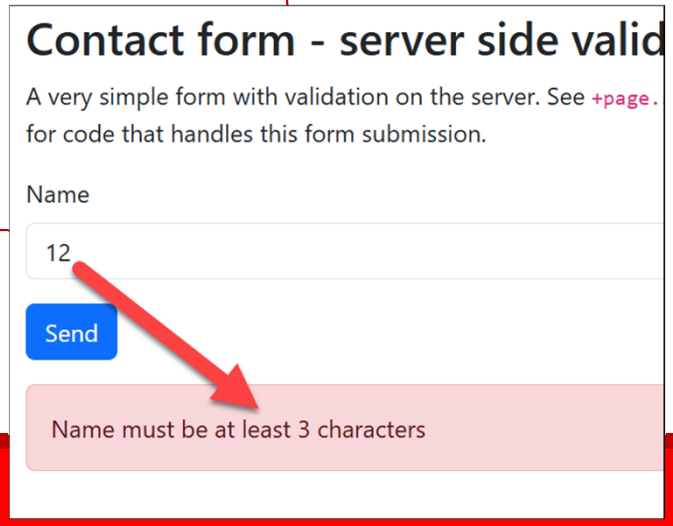
- `request.formData()` → we get form values from the client, we can work with it server-side
- `return` value → becomes `form` `$props()` in the page
- no `fetch()` needed

Server side validation

- In addition to client side validation, we can validate a request on the server. For instance:

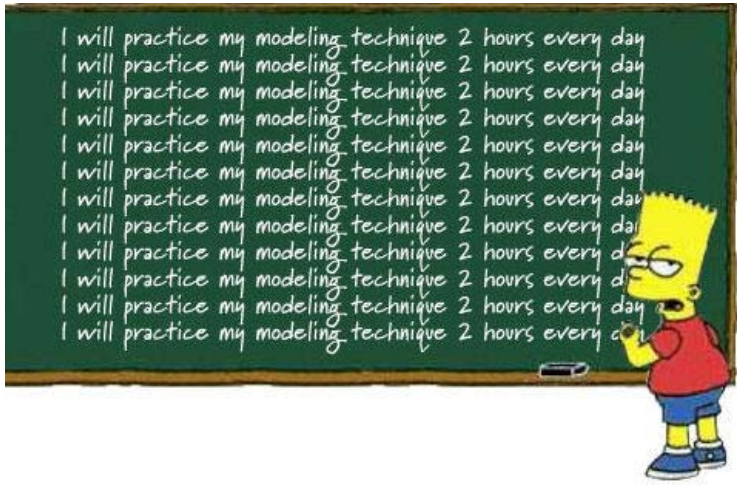
```
const data = await request.formData();
const name = data.get('name');

// test if name is of type string, otherwise there is no
'length' property and we get a TS error.
if (typeof name !== 'string' || name.trim().length < 3) {
  console.log('ERROR name was to short ::', name);
  return {
   error: 'Name must be at least 3 characters'
  };
}
```



Workshop

- Create an application/component with a simple form
- Handle form submission on the server.
- For instance: a `login` form with 2 fields
  - `name`, `password`.
- Return `{ login: true | false }` based on simple rules you make up yourself
- Example: `…/470-sveltekit-form-action`



Multiple Actions

A single `+page.server.ts` can have multiple actions to react upon

File `+page.server.ts` can have more actions

- See `/form03` and `+page.server.ts`, exposing a `login` and `register` function
- These are called from the form, with `method="POST"` `action="?/login` or `action="?/register`.
- Handle all logic on the server in TypeScript

SvelteKit Demo Home Form 01 - simple Form 02 - validation **Form 03 - multiple actions** Form 04 - calling an API About

Multiple forms - multiple actions

This page has two forms, it also has two actions in the backend `+page.server.ts`. They are annotated with `?/login` and `?/register`

Form 1 - Login

Name

Password

Login

This form has just an email field and a `Register` button

Form 2 - Register

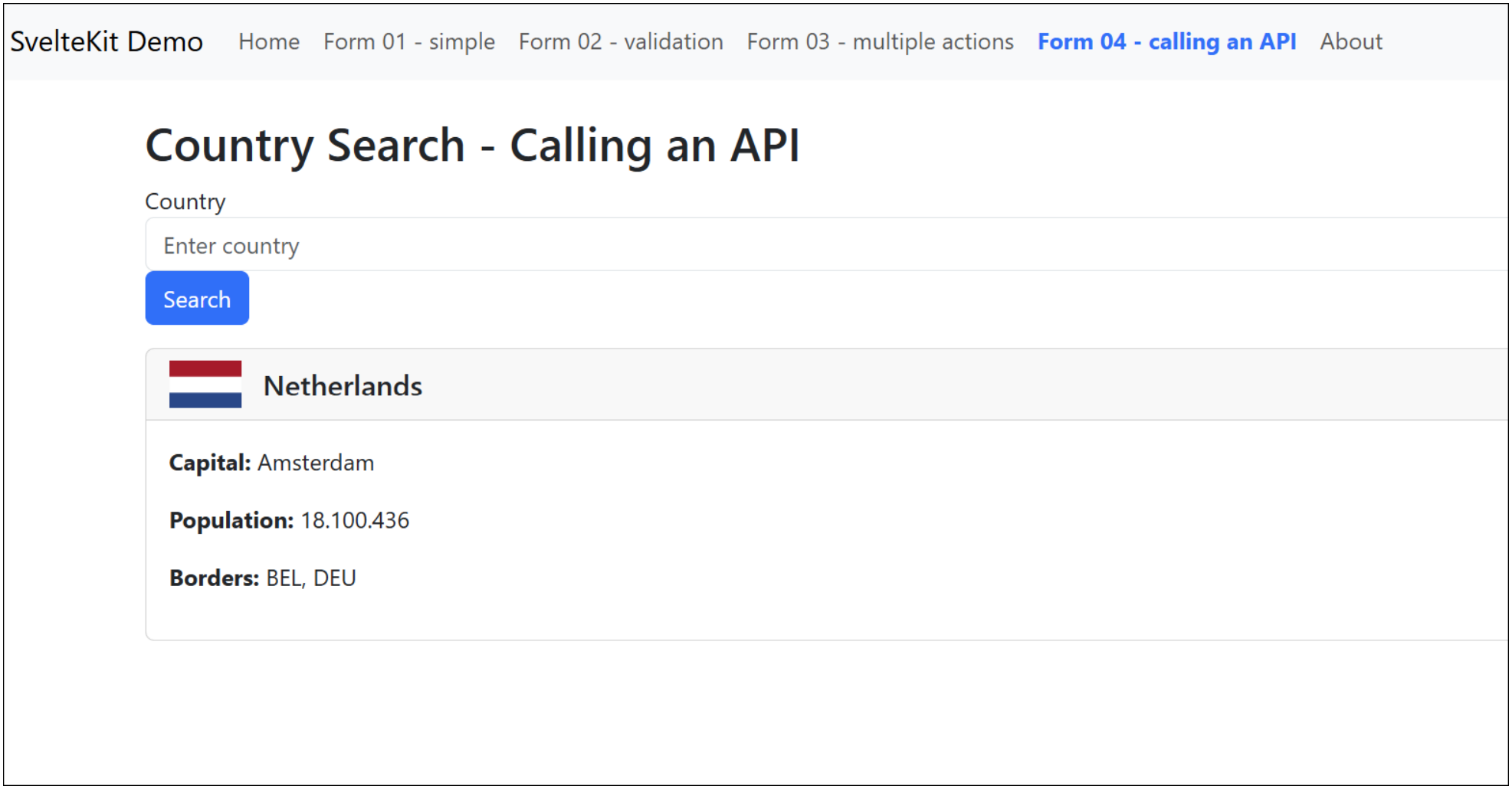
Email

Your email

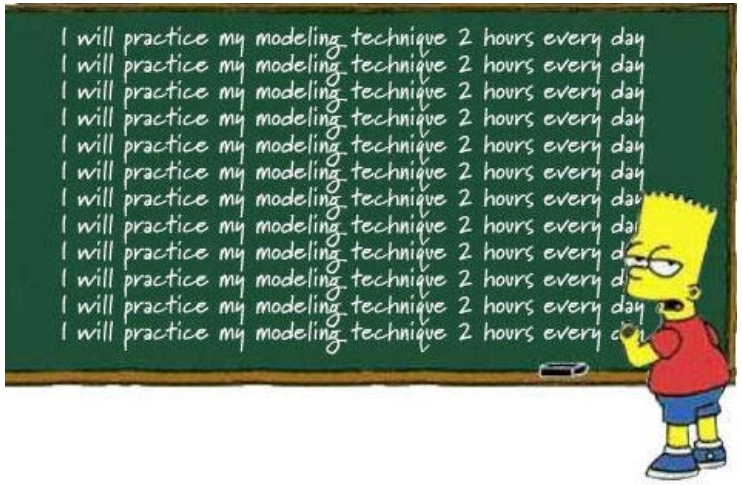
Register

../470-sveltekit-form-actions/form03

Custom form actions + API fetch()



../470-sveltekit-form-actions/form04

Workshop

- TBD...
- Example: `…/470-sveltekit-form-actions`